ELSEVIER

Procedia Computer Science 00 (2026) 1–13

Journal Logo

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

Dropout-Aware Training for Sensor-Robust Fault Localization with Spatio-Temporal Graph Neural Networks

P K Sangameswar[a], Dr. M Anbazhagan[a,*]

[a]*Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore, Amrita Vishwa Vidyapeetham, India*

Abstract

Accurate fault localization, identifying where a fault occurs and not merely that one has occurred, is essential for reducing outage duration in power distribution systems, yet remains difficult under partial observability, where only a few micro-phasor measurement units (μPMUs) monitor networks of hundreds of buses. Spatio-temporal graph neural networks (STGNNs) localize accurately in this setting, but their evaluations assume every μPMU is always available, an assumption that fails in deployed systems, where sensors are lost to communication faults, maintenance, or hardware failure. Following the STGNN methodology of [1] on the IEEE 123-bus feeder, we build our own dataset and train GRU, RGCN, RGSAGE-Mean, RGSAGE-Max, and RGATv2 models; our strongest reaches a soft-voting macro-F1 of 0.85, our reference point. We then present the first systematic evaluation of STGNN robustness to μPMU loss, dropping $k \in \{0, 1, 2, 5, 10\}$ of the 25 sensors at inference: the strongest baseline falls from 0.85 to below 0.10 at 40% loss, and clean accuracy proves a poor predictor of robustness, since the most accurate architecture degrades the fastest. To mitigate this we introduce dropout-aware training, an architecture-agnostic scheme that masks measurement nodes during training so the model depends on no particular sensor, at no inference-time cost. It raises the retained macro-F1 at 40% loss from 5.7% to 80.3% of the full-sensor score at virtually no cost to clean performance (ΔF1 < 0.01 at $k = 0$), and the gain holds across all five architectures. An availability-gated architectural variant matches but does not surpass this gain, indicating that sensor-dropout robustness is best addressed during training rather than through the architecture.

Keywords: Fault localization, Spatio-temporal graph neural networks, Sensor dropout robustness, Missing measurements, Dropout-aware training, Distribution grids

1. Introduction

Rapid and accurate fault localization is essential for automated restoration in modern power distribution grids. As feeders absorb growing shares of distributed energy resources and adopt self-healing schemes, how precisely a system identifies where a fault occurred, not merely that one occurred, determines how much of the network can be re-energized before crews are dispatched. Traditional impedance-based and traveling-wave methods assume densely deployed measurement devices, which does not hold when micro-phasor measurement units (μPMUs) are installed at only a few locations because they are expensive and need communication infrastructure. A feeder with hundreds of buses may carry only a handful of μPMUs, so any fault-localization method must work under partial observability.

*Corresponding author. Tel.: +91-7708543294.
Email address: m_anbazhagan@cb.amrita.edu (Dr. M Anbazhagan)

Table 1. Positioning of representative fault-localization methods (D = detection, C = type classification, L = localization; ✓ yes, – not studied, ~ partial).

Work	Approach / model	Task	Test system	Multi-arch.	Sparse obs.	Drop-out @ test	Loss mitigation
Model-based							
Krishnathevar & Ngu (2012) [4]	Generalized impedance-based	L	Distribution	–	–	–	–
Liu et al. (2023) [5]	Traveling wave, multi-branch	L	Distribution	–	–	–	–
Non-graph data-driven							
Livani et al. (2013) [6]	SVM, faulty-line identification	L	IEEE 123-bus	–	–	–	–
Static GNN							
Chen et al. (2020) [7]	Spectral GCN (Chebyshev)	L	IEEE 123-, 37-bus	–	~	✓	✓
Fan et al. (2024) [3]	VGAE + GraphSAGE	L	Distribution	–	–	–	–
Spatio-temporal GNN							
Nguyen et al. (2022) [8]	1D-CNN + GCN	D+C+L	Microgrid	–	–	–	–
Ngo et al. (2025) [9]	GAT + 1D-CNN	D+C+L	Distribution	–	–	–	–
Sparse / missing measurement							
Zhang et al. (2025) [2]	GDIA-GCL: imputation + contrastive	L	Distribution	–	✓	~	✓
Base paper and this work							
Karabulut et al. (2026) [1]	STGNN: RGCN, RGSAGE, RGATv2	D+L	IEEE 123-bus	✓	✓	–	–
This work	**STGNN + node drop-out study**	**D+L**	**IEEE 123-bus**	✓	✓	✓	✓

Graph neural networks (GNNs) are a natural fit: the feeder's known topology can be encoded so that message passing shares information between connected buses, letting the model infer faults even at buses carrying no sensor. Building on this, [1] introduced a spatio-temporal GNN (STGNN) coupling a GRU temporal encoder with a spatial GNN (RGCN, RGSAGE, or RGATv2) over a *measured-only* graph in which only instrumented buses are nodes, predicting one of 26 classes (25 fault locations plus no-fault). We adopt their framework and evaluation methodology as the basis for our robustness study.

That result, however, rests on an assumption that does not hold in deployed systems: that every μPMU is always available and functioning. Real sensors drop out through communication loss and hardware failure, and a model trained and evaluated only with a complete sensor set carries no guarantee of degrading gracefully when some inputs go dark. Prior work on incomplete measurement addresses a different regime: Zhang et al. [2] impute chronically unmeasured buses via graph Dirichlet imputation before contrastive GCN training, and Fan et al. [3] target measurement noise through a variational graph autoencoder, but both assume a sensor-availability pattern fixed and known at training time. To the best of our knowledge, no previous work has studied how STGNN-based fault localization is affected when sensors fail during inference.

We therefore re-implement the four STGNN baselines of [1] (RGCN, RGSAGE-Mean, RGSAGE-Max, RGATv2) and their temporal-only GRU reference, train them on our own regenerated dataset under a common protocol whose full-observability accuracy is our reference point (Section 3.3), and make three contributions. First, we quantify how far fault-localization accuracy degrades when $k \in \{0, 1, 2, 5, 10\}$ μPMUs are dropped at inference time, across all five architectures, producing macro-F1-vs-k degradation curves. Second, we introduce dropout-aware training, a simple, architecture-agnostic *training method* that randomly masks a variable number of measurement nodes during training, and show it recovers a substantial fraction of the accuracy lost to sensor dropout. Third, as an *architectural* counterpart we introduce *SDR-STGNN* (Sensor-Dropout-Robust STGNN), an availability-gated network built on RGSAGE-Max that structurally silences dropped μPMUs during message passing, and evaluate whether it improves robustness beyond dropout-aware training alone (Section 3.6). All comparisons use identical data, splits, and evaluation protocol (Section 3.3).

2. Related Work

Early approaches to fault localization in distribution grids estimate the fault position from circuit physics. Impedance-based methods infer distance from the apparent impedance seen at a measurement point; Krishnathevar and Ngu [4] give a generalized one-end formulation that resolves the multiple-estimation ambiguity, in which a single measured impedance maps to several candidate positions on a branching feeder. Travelling-wave methods instead time the fault-induced wavefront: Liu et al. [5] sharpen the arrival instants with an improved complete ensemble empirical mode decomposition and a Teager energy operator, then

discriminate among branches by arrival-time differences. Data-driven alternatives relax the need for an exact circuit model, as in Livani et al. [6], who identify the faulted line of the IEEE 123-node feeder with support-vector machines over post-fault RMS voltages and fault-current-indicator and recloser signals. All three assume measurement coverage dense enough to observe the relevant impedances or wavefronts, and none represents which buses are electrically adjacent. Table 1 positions them, and the work below, along the axes that matter here: multi-architecture comparison, sparse observability, evaluation under test-time sensor failure, and mitigation of the resulting loss.

Encoding the feeder as a graph restores the structure that feature-vector models discard. Chen et al. [7] apply a spectral graph convolutional network with Chebyshev filters to the IEEE 123- and 37-bus systems, integrating measurements from several buses while respecting system topology, and report higher accuracy than support-vector, random-forest, and fully connected baselines. Their data-augmentation procedure, which hardens the model against measurement noise and data loss and lets it operate with few measured buses, is the closest precedent for the training-time mitigation we study, although it is applied to a static, single-snapshot classifier over the full grid graph. Fan et al. [3] pursue reconstruction instead, pairing a variational graph autoencoder with GraphSAGE so that the autoencoder recovers latent node information and improves tolerance to measurement noise. Both classify a single snapshot, so neither represents how a fault signature evolves over the milliseconds following inception.

Spatio-temporal models supply that missing time axis by placing a temporal encoder ahead of message passing. Nguyen et al. [8] and Ngo et al. [9] both adopt this composition, using a one-dimensional CNN with a GCN and with graph attention respectively, and both cover detection, fault type and phase classification, and localization within a single framework (Table 1). This paper builds directly on Karabulut et al. [1], who benchmark RGCN, RGSAGE, and RGATv2 spatial encoders on a shared GRU temporal encoder over the IEEE 123-bus feeder. Their central construction is the *measured-only* graph, in which only instrumented buses become nodes, so that message passing operates over the sensing infrastructure rather than the physical feeder; it outperforms a purely recurrent baseline by up to 11 F1 points and cuts training time roughly sixfold against building the graph over the full network. Treating sparse μPMU placement as a first-class property of the model rather than a preprocessing detail makes it the natural foundation for asking what happens when the placed sensors themselves fail. A companion study [10] examines a different robustness axis, how far such detectors transfer to grid configurations other than the one they were trained on.

A separate line of work confronts incomplete measurement directly. Zhang et al. [2] address distribution networks in which a large fraction of bus measurements is simply absent: a graph Dirichlet imputation algorithm first reconstructs the missing values by minimizing the graph Dirichlet energy, a training-free procedure requiring no assumption about the underlying data distribution, and a GCN trained with graph contrastive learning then keeps fault signatures separable under the resulting scarcity. This is the most developed treatment of missing data in this task, and it is instructive for what it assumes: that the set of unmeasured buses is a fixed property of the network, known when both the imputer and the classifier are built.

Read together, these lines of work leave the two right-hand columns of Table 1 almost empty, and that emptiness defines this paper's opening: no prior method combines sparse observability with an evaluation of sensors that fail *after* deployment and a mitigation for the accuracy lost when they do. The imputation and reconstruction approaches [2, 3] come closest, but both assume a missingness pattern fixed and known at training time, which is precisely what an inference-time failure is not: the identity of the lost sensors is unknown when the model is built and differs from one outage to the next. Chen et al. [7] do augment against data loss, but for a static, full-graph GCN, so their result does not establish how a spatio-temporal model behaves when whole nodes of an already sparse μPMU set go dark. Karabulut et al. [1] handle sparse placement but assume every placed sensor always reports, and [10] varies the grid rather than the sensor set. That assumption is the one most likely to fail first in practice: a μPMU depends on communication infrastructure and periodic maintenance, so on a feeder instrumented with 25 of them, losing a handful is an ordinary operating event. Our objective follows: to measure how far STGNN localization falls as μPMUs are lost at inference, across architectures rather than for a single model, and to establish what recovers it, separating what can be fixed by training from what requires changing the model itself.

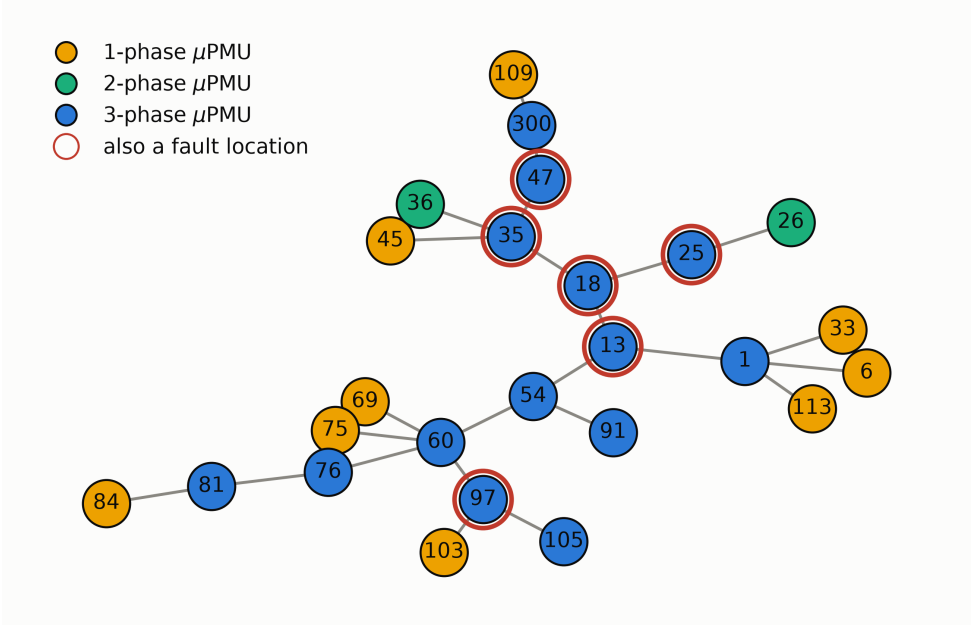


Fig. 1. The measured-only graph of 25 μPMU buses (coloured by phase count) and 24 topology-derived edges, with red rings on the 6 buses that are also fault locations.

3. Materials and Methods

3.1. Grid Simulation and Dataset

We generate our own corpus on the IEEE 123-bus distribution feeder, a standard radial benchmark, simulated in OpenDSS through the OpenDSSDirect.py interface. Simulations run in daily (quasi-static time-series) mode, so the loads evolve along a predefined profile and the recorded signals capture fault dynamics rather than steady-state snapshots. Table 2 gives the full simulation configuration, feature layout, and per-split window counts, every entry computed directly from the generated dataset.

Partial observability is modeled by instrumenting only 25 buses with μPMUs, following the placement of [1] (single-, two-, and three-phase units reporting three-phase RMS voltages). A reduced *measured-only* graph is built from these buses (Fig. 1): the 25 μPMUs are its nodes and their connectivity is inherited from the feeder topology, giving a connected tree with 24 edges, so message passing operates on the sensing infrastructure rather than the complete physical feeder. Only 6 of the 25 candidate fault locations coincide with an instrumented bus; the remaining 19 are never directly observed, the central difficulty the spatial model must overcome.

Two encoding choices behind Table 2 carry methodological weight. First, the availability mask defines what "missing" means throughout this paper: $m_p = 1$ marks an available measurement and $m_p = 0$ a missing phase or a dropped sensor, so a μPMU that fails is encoded exactly like one that was never installed. Second, the 70/15/15 partition is taken at the *event* level, so the 40 sliding windows of one event never span more than one split and no window of a test event is seen during training. Voltage channels are z-score normalized using training-split statistics only. We adopt the base paper's dataset-generation methodology but build our own corpus (Section 3.3); since every model is trained and evaluated on it with identical splits, the comparisons remain fair. Fault severity varies sharply with fault resistance (Fig. 2): for a three-phase-to-ground fault at bus 101, the deepest-sagging μPMU channel drops 0.83 p.u. below its pre-fault level at 0.1 Ω, 0.22 p.u. at 1 Ω, and only 0.01 p.u. at 10 Ω, which is why high-impedance faults are the hardest class to localize throughout this study.

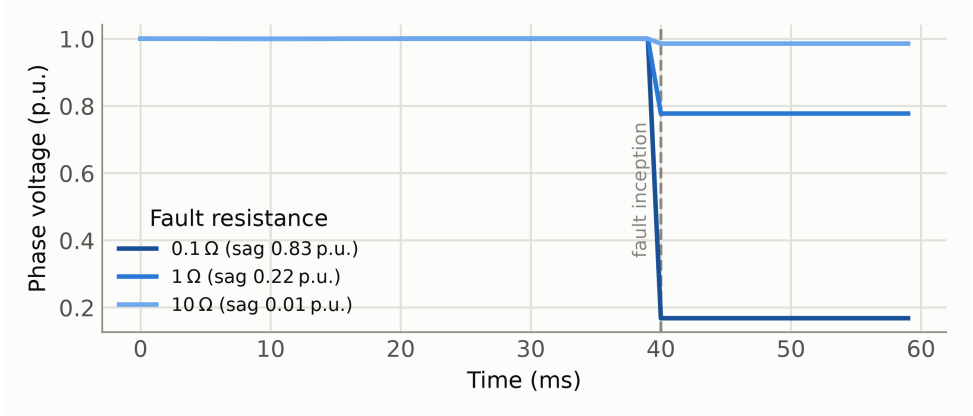


Fig. 2. Deepest-sagging μPMU channel for a three-phase-to-ground fault at bus 101, normalized to its pre-fault mean, showing that sag depth shrinks as fault resistance rises.

Table 2. Dataset configuration and event-level 70/15/15 split of the 50-run corpus.

Simulation configuration	
Test feeder / simulator	IEEE 123-bus, 4.16 kV, 60 Hz; OpenDSS (OpenDSSDirect.py), Daily mode, 1 ms step
Fault buses / μPMUs	25 / 25 (single-, two-, and three-phase units)
Fault types / resistances	11 (3 LG, 3 LL, 3 LLG, 2 three-phase) / 3 (0.1, 1, 10 Ω)
Load multiplier	$\mathcal{U}(0.5,\ 1.3)$ per run
Scenarios × runs	$25 \times 11 \times 3 = 825$ scenarios × 50 runs
Record / windowing	60 ms record (40 ms pre-fault); 20-step (20 ms) windows, 40 per event
Node features F / classes	6 ($[V_1, V_2, V_3]$ + availability mask $[m_1, m_2, m_3]$) / 26 (25 locations + no-fault)

Window-level distribution (event-level split)	
Split	Fault / No-fault / Total windows (events)
Train	577,500 / 577,500 / 1,155,000 (28,875)
Validation	124,000 / 124,000 / 248,000 (6,200)
Test	123,500 / 123,500 / 247,000 (6,175)
Total	825,000 / 825,000 / 1,650,000 (41,250)

3.2. Baseline Architectures: GRU, RGCN, RGSAGE, RGATv2

We evaluate the four STGNN architectures of [1] (RGCN, RGSAGE-Mean, RGSAGE-Max, and RGATv2) together with the temporal-only GRU that serves as their non-spatial reference; the GRU has no message-passing stage and is therefore not itself a spatio-temporal GNN. All five follow the designs of [1] unchanged, the one exception being spatial depth, a configuration choice discussed below. On top of the strongest baseline, RGSAGE-Max, this work adds a training method, *dropout-aware training* (Section 3.5), and an architecture, *SDR-STGNN*, evaluated as an ablation against it (Section 3.6). All five share a common template: a temporal encoder produces one embedding per μPMU node, an optional spatial stage exchanges information over the measured-only graph, and a shared per-node head with soft voting yields the graph-level prediction. Let $x_v \in \mathbb{R}^{S\times F}$ be the input at μPMU node v of the $N = 25$ node measured-only graph, over the $S = 20$ step window with $F = 6$ channels, and let $\mathcal{N}(j)$ denote the neighbours of node j in that graph. A single-layer GRU with 128 hidden units, shared across nodes and applied to each node independently, gives the node embedding of Eq. (1):

$$h_v^{(0)} = \mathrm{GRU}(x_v) \in \mathbb{R}^{128}, \qquad v = 1, \ldots, N. \tag{1}$$

The GRU-only baseline stops at Eq. (1) and cannot exploit spatial relationships. The four GNN variants stack $L = 4$ message-passing layers (hidden dimension 64, ReLU between layers, self-loops added) over that graph and differ only in the aggregation operator: *RGCN* propagates over the symmetrically normalized

adjacency (Eq. (2)); *RGSAGE-Mean* and *RGSAGE-Max* concatenate a node's own embedding with a mean- or max-pooled neighbourhood before projecting (Eq. (3)) [11]; and *RGATv2* applies $H = 4$ head GATv2 attention over each node's in-neighbours (Eq. (4)) [12]:

$$h^{(l+1)} = \mathrm{ReLU}\Big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}\, h^{(l)} W^{(l)}\Big), \tag{2}$$

$$h_j^{(l+1)} = \mathrm{ReLU}\Big(W^{(l)}\Big[h_j^{(l)} \,\Big\|\, \rho(\{\, h_i^{(l)} : i \in \mathcal{N}(j)\,\})\Big]\Big), \tag{3}$$

$$h_j^{(l+1)} = \mathrm{ReLU}\left(\Big\|_{c=1}^{H} \sum_{i\in\mathcal{N}(j)\cup\{j\}} \alpha_{ji}^{(l,c)}\, W^{(l,c)} h_i^{(l)}\right), \tag{4}$$

where $\tilde{A} = A + I$ is the adjacency with self-loops and $\tilde{D}$ its degree matrix, $W^{(l)}$ are the layer weights, $\|$ is concatenation, ρ is mean or max pooling, and $\alpha_{ji}^{(l,c)}$ is the GATv2 attention coefficient of head c. These operators are adopted unchanged from [1].

Spatial depth (configuration choice): The base paper does not state how many message-passing layers its GNN variants use, so the depth had to be selected here. A single hop gave substantially weaker localization, because most fault locations lie more than one hop from any μPMU (only 6 of 25 fault buses are instrumented); we therefore stack *four* message-passing layers in every GNN variant, fixed for all experiments (Section 4).

Classification and aggregation: A shared head (batch normalization, ReLU, dropout $p = 0.35$, linear projection to 26 classes) maps the final embedding of every node to logits z_v. At inference the per-node class probabilities are summed over all N nodes and the arg-max is the graph-level prediction (soft voting, Eq. (5)). Training minimizes the unweighted node-level cross-entropy of Eq. (6), in which each window's event label y supervises all nodes; class balance is handled by the balanced corpus (Section 3.7) rather than by loss weighting.

$$\hat{y} = \arg\max_{c\in\{0,\ldots,25\}} \sum_{v=1}^{N} \mathrm{softmax}(z_v)_c, \tag{5}$$

$$\mathcal{L} = \frac{1}{N}\sum_{v=1}^{N} \mathrm{CE}(z_v,\, y)\,. \tag{6}$$

3.3. Deviations from the Base Setup

Our setup follows [1] (feeder, μPMU placement, fault taxonomy, windowing, soft-voting evaluation, and an unweighted cross-entropy objective) but differs in four documented respects: a spatial depth of four message-passing hops, which the base paper leaves unspecified; a 1.65 M-window corpus against its 2.5 M; a longer schedule of 30 epochs with gradient clipping and learning-rate warmup, against its 15 epochs for the GNN models and 11 for the GRU; and omission of the reconfigured ("green") topology. All comparisons here use our own corpus and splits, so they are internally consistent and should be read against our own 0.85 reference point rather than the base paper's absolute scale.

3.4. Sensor-Dropout Evaluation Protocol

We evaluate robustness to μPMU failures entirely at *inference* time, with all model parameters fixed, which isolates robustness from any training-time mitigation.

Missingness model: To simulate a lost sensor we select k of the 25 μPMUs and, for every test window, zero all six input channels of each selected node, both the RMS-voltage and the availability-mask channels. Zeroing the mask is essential: a zero voltage can be a valid reading, so the mask is the definitive indicator that a node is unobserved. This matches the encoding of absent sensors in the dataset (Section 3), making a failed μPMU indistinguishable from one that was never instrumented.

Algorithm 1 Dropout-aware training; only the per-batch masking of lines 4–5 differs from standard training.

Require: training batches $\mathcal{B}$; dropout levels $\mathcal{K} = \{0, 1, 2, 5\}$; epochs E; model f_θ
Ensure: dropout-aware weights θ

```
1: for e = 1 to E do
2:     reseed the batch-masking RNG                          ▷ fresh dropout each epoch
3:     for each mini-batch (x, y) ∈ B, x ∈ R^{B×N×S×6} do
4:         k ← Uniform(K)                                    ▷ one level per batch
5:         for every window b ≤ B, draw k nodes uniformly and zero all six channels of x_b at those nodes
6:         Z ← f_θ(x);  L ← mean of Eq. (6) over the batch   ▷ Z ∈ R^{B×N×26}
7:         backpropagate; clip gradient norm to 1.0; AdamW step   ▷ LR warmup over first 3 epochs
8:     end for
9: end for
```

Graph handling: A dropped node keeps its edges and still participates in message passing, but its temporal encoder receives an all-zero sequence and emits a zero-valued embedding that propagates to neighbours, exactly the non-informative message the availability-gated model (Section 3.6) is designed to suppress. All models see the same graph, so differences arise solely from the message-passing mechanism. (An edge-removal policy was implemented but not used, to avoid confounding topology with dropout.)

Dropout sweep and metrics: We sweep $k \in \{0, 1, 2, 5, 10\}$ (0/4/8/20/40% loss), with $k = 0$ the full-observability reference. Dropped sensors are sampled uniformly at random per window, driven by fixed seeds reused across all models so that every architecture sees an identical sequence of dropout realizations on the same frozen test split (Table 2). Each (model, k) point is averaged over $n_{\text{seed}} = 3$ mask realizations and, where stated, over training seeds 42/43/44. Performance is the soft-voted macro-F1 over the 26 classes, giving a macro-F1-versus-k *degradation curve*, from which we derive the *retention* $R(k)$ of Eq. (7) and the *normalized area under the retention curve* (AUDC) of Eq. (8):

$$R(k) = \frac{\text{F1}(k)}{\text{F1}(0)}, \tag{7}$$

$$\text{AUDC} = \frac{1}{k_{\max} - k_{\min}} \int_{k_{\min}}^{k_{\max}} R(k)\, dk \in [0, 1]. \tag{8}$$

Here $k_{\min} = 0$ and $k_{\max} = 10$ are the extremes of the sweep, and Eq. (8) is computed by trapezoidal integration over $k \in \{0, 1, 2, 5, 10\}$; a higher AUDC means slower degradation. Both metrics are relative to a model's own clean score, so they compare how gracefully models degrade, not how accurate they are.

3.5. Dropout-Aware Training

Dropout-aware training removes the train-test mismatch exposed in Section 3.4: the baselines train with all 25 μPMUs available but are evaluated under partial observability. We apply the evaluation missingness model as training-time augmentation, drawing a dropout level $k \sim \text{Uniform}\{0, 1, 2, 5\}$ per batch and zeroing k randomly chosen nodes before each forward pass (Algorithm 1). Because k varies batch to batch, every epoch mixes full- and partial-observability cases, so no μPMU is consistently available and the network cannot depend on any individual sensor. The augmentation touches only the input features, not the graph, architecture, or optimizer, so it applies unchanged to all baselines and to SDR-STGNN (Section 3.6). Masking graph elements during training is itself a known regularizer (e.g., DropEdge [13]); our contribution is not the primitive but framing μPMU dropout as a train-test mismatch and studying its mitigation.

All other training details are identical to the baseline (Section 3.7), so any robustness difference is attributable solely to the augmentation, which is applied only during training and adds no parameters or inference-time cost.

3.6. SDR-STGNN: Availability-Gated Graph Aggregation

SDR-STGNN (Sensor Dropout-Robust Spatio-Temporal Graph Neural Network) addresses sensor dropout architecturally and is included as an ablation against dropout-aware training. All five baselines receive the availability mask as input channels $[m_1, m_2, m_3]$, yet none of the operators in Eqs. (2)–(4) uses that information, so a dropped node keeps contributing its zero-valued embedding as a non-informative message. SDR-STGNN makes the aggregation of the strongest baseline, RGSAGE-Max, *availability-gated*, so dropped sensors send no messages at all.

Availability: For each input window we derive a binary availability indicator for every node from its three availability-mask channels, given by Eq. (9):

$$a_i = \mathbf{1}\left\{ \max_{s} \max_{p\in\{1,2,3\}} m_{i,p}(s) > 0 \right\} \in \{0, 1\}, \tag{9}$$

where $m_{i,p}(s)$ is the mask value of phase p at node i and step s; thus $a_i = 1$ iff any mask channel is active at any step in the window, so both dropped μPMUs and never-instrumented locations receive $a_i = 0$.

Gated aggregation: Eq. (10) replaces the RGSAGE update of Eq. (3) at each of the four message-passing layers, writing h_i for $h_i^{(l)}$ and h'_j for $h_j^{(l+1)}$: the max-aggregation is restricted to available source nodes, and the self-representation is gated by the same availability indicator:

$$h'_j = \mathrm{ReLU}\Big(W\left[a_j h_j \,\|\, \mathrm{agg}_j\right]\Big), \qquad \mathrm{agg}_j = \max_{\substack{i\in\mathcal{N}(j)\cup\{j\}\\ a_i=1}} h_i, \tag{10}$$

where W is the layer projection of Eq. (3) and $\mathcal{N}(j)$, $\|$ are as defined there. Nodes with $a_i = 0$ send no messages, and $\mathrm{agg}_j = \mathbf{0}$ when the entire closed neighbourhood of j is unavailable. The availability indicators are computed once from the input window (Eq. (9)) and reused across all four layers. The gate adds no parameters, and the shared GRU encoder, four-hop depth, classifier head, and soft voting are identical to RGSAGE-Max; when every sensor is available ($a_i = 1$ for all i), Eq. (10) reduces exactly to the standard RGSAGE-Max update, so the two are identical under full observability ($k = 0$). SDR-STGNN is trained with the dropout-aware augmentation of Section 3.5, so the gates are exercised throughout training.

Scope of the ablation: On the same RGSAGE-Max backbone we compare three variants: (i) standard training; (ii) dropout-aware training (ungated, mask as input only); and (iii) SDR-STGNN (dropout-aware training plus availability-gated aggregation). Since everything else is identical, the comparison separates the training-time augmentation from the structural gate (Section 4).

3.7. Experimental Setup

All models share one training and evaluation configuration, listed in Table 3. Apart from the variable under study (the message-passing architecture or the augmentation of Section 3.5), every part of the pipeline is fixed, so any difference is attributable to that variable alone. The points not covered by Table 3 are:

- *Batching:* an event-chunk sampler keeps all sliding windows of an event together, consistent with the event-level partition of Table 2.
- *Objective:* imbalance is handled by dataset construction rather than loss weighting. The corpus is 50% no-fault and the $25 \times 11 \times 3$ factorial design represents the 25 fault locations uniformly, so inverse-frequency weighting is unnecessary, matching the unweighted objective of [1].
- *Determinism:* parameter initialization, mini-batch ordering, and the dropout-aware masking are all controlled by one integer seed; no early stopping is used beyond best-validation checkpointing.
- *Evaluation:* each trained model is frozen and swept over k under the protocol of Section 3.4, with identical fixed mask seeds across models.
- *Availability:* the implementation is available from the authors upon reasonable request.

Table 3. Training, evaluation, and software configuration shared by all models.

Model configuration	
Temporal encoder	1-layer GRU, 128 hidden units, shared across nodes
Spatial layers	4 message-passing layers, hidden dimension 64
Classifier head	BatchNorm → ReLU → dropout 0.35 → linear (26); attention dropout 0.3 (RGATv2)
Graph-level readout	Soft voting over the 25 node predictions
Optimization	
Optimizer	AdamW, learning rate 10^{-3}, weight decay 10^{-4}; batch size 32, 30 epochs
Schedule / clipping	Linear LR warmup over first 3 epochs, constant thereafter; global norm 1.0
Loss / model selection	Node-level unweighted cross-entropy; best validation graph macro-F1
Evaluation and reporting	
Training seeds	42, 43, 44 (mean ± std reported)
Dropout levels k	0, 1, 2, 5, 10 of 25 μPMUs (0–40% loss)
Mask realizations	3 per training seed, seeds shared across models
Metrics	Soft-voted macro-F1, retention, AUDC
Hardware and software	
GPU	1 × NVIDIA T1000 (8 GB)
Software stack	Python 3.12, PyTorch 2.5.1 (CUDA 12.1), PyTorch Geometric 2.8, OpenDSSDirect.py 0.9.4, NumPy 2.5, scikit-learn 1.9

4. Results

We first establish the baselines under full observability, then quantify how those fixed models degrade as μPMUs are removed at inference (the paper's primary evaluation), then compare dropout-aware training against the availability-gated SDR-STGNN, and finally test whether the training-time fix generalizes across architectures. Unless otherwise stated, results are computed on the frozen test split of Table 2 and reported as mean ± standard deviation over training seeds 42, 43, and 44, consistent with Section 3.7.

4.1. Baseline Accuracy Under Full Observability

Table 4 summarizes the five baselines under full observability ($k = 0$); the two macro-F1 columns carry ± one standard deviation over seeds 42/43/44, while the per-resistance columns are seed means. The four spatio-temporal GNN models consistently outperform the temporal-only GRU: RGSAGE-Max achieves the highest graph-level macro-F1 of 0.846 ± 0.007, followed by RGSAGE-Mean (0.786 ± 0.023) and RGCN (0.712 ± 0.017), whereas the GRU attains only 0.286 ± 0.009. That gap shows the importance of spatial message passing here: because only 6 of the 25 fault locations are instrumented, localization usually requires information to propagate from instrumented buses to unmeasured fault locations (Section 3), and Eq. (1) alone provides no mechanism for that. For every model the graph-level macro-F1 exceeds the node-level macro-F1, so the soft vote of Eq. (5) consistently improves on individual node predictions.

Accuracy decreases monotonically as fault resistance increases, for every model: RGSAGE-Max, for example, drops from 0.949 at 0.1 Ω to 0.676 at 10 Ω, reflecting the weaker voltage signatures of high-impedance faults (Section 3). Fault detection, in contrast, is effectively saturated: all models achieve a no-fault recall of at least 0.9999 and a no-fault precision of at least 0.972, so the primary challenge is localization rather than detection. Unlike [1], RGATv2 is the weakest GNN model here (0.522 ± 0.087) and the highest in variance across seeds; that spread reflects training instability at the adopted four-layer depth, with metrics taken from the best validated checkpoint of each seed without extensive hyperparameter tuning. Stabilizing RGATv2 at that depth is left as future work.

These results also justify the spatial depth of four: a single-seed diagnostic depth sweep on RGSAGE-Max rises monotonically from 0.509 (1 hop) to 0.806 (4 hops, reduced budget), with the full 30-epoch recipe reaching 0.846 and no sign of over-smoothing. Because only 6 of the 25 fault buses are instrumented, a single hop cannot reach faults two or more hops from any μPMU, so the enlarged receptive field is the primary driver of accuracy.

Table 4. Baseline macro-F1 under full observability, as mean±std over seeds 42/43/44, with the best value per column in bold.

Model	Graph-level	Node-level	Macro-F1 by fault resistance		
	Macro-F1	Macro-F1	0.1 Ω	1 Ω	10 Ω
RGSAGE-Max	**0.846 ± 0.007**	**0.718 ± 0.008**	**0.949**	**0.893**	**0.676**
RGSAGE-Mean	0.786 ± 0.023	0.660 ± 0.025	0.906	0.843	0.587
RGCN	0.712 ± 0.017	0.601 ± 0.019	0.874	0.792	0.428
RGATv2	0.522 ± 0.087	0.460 ± 0.080	0.630	0.574	0.312
GRU	0.286 ± 0.009	0.150 ± 0.003	0.410	0.292	0.105

Table 5. Degradation under sensor dropout for standard (Std) and dropout-aware (DA) training, averaged over seeds 42/43/44 with three mask realizations each.

Model	Standard-training macro-F1 at k					AUDC		Ret.@10	
	k=0	k=1	k=2	k=5	k=10	Std	DA	Std	DA
RGATv2	0.522	0.490	0.444	0.302	0.157	**0.621**	**0.960**	**30.1%**	**91.3%**
RGCN	0.712	0.664	0.610	0.418	0.164	0.607	0.940	23.1%	87.0%
RGSAGE-Mean	0.786	0.720	0.647	0.393	0.108	0.540	0.921	13.8%	81.7%
GRU	0.286	0.273	0.252	0.111	0.051	0.521	0.784	17.9%	25.8%
RGSAGE-Max	**0.846**	**0.736**	0.590	0.236	0.048	0.402	0.913	5.7%	80.3%

4.2. Accuracy Degradation Under Sensor Dropout

We now freeze each baseline and evaluate it under the dropout sweep of Section 3.4. Table 5 summarizes the degradation curves through AUDC (Eq. (8)) and retention at the most severe level ($k = 10$, 40% sensor loss); Fig. 3 shows the full curves. Every model declines as sensors are removed, and no baseline retains more than a third of its full-observability macro-F1 at $k = 10$.

The results reveal a clear trade-off between clean accuracy and robustness; the training-seed standard deviation stays below 0.06 at every k for four of the five models and reaches 0.09 for RGATv2. RGSAGE-Max, the strongest model under full observability, is the least robust, retaining only 5.7% of its clean macro-F1 at $k = 10$, while the remaining models retain a larger fraction of their (lower) baseline accuracy, RGATv2 and RGCN highest. RGATv2's nominal top-robustness position is indicative only: it has the lowest clean accuracy of the GNNs and the largest seed variance (±0.087), so its high *relative* retention partly reflects a low starting point. The robust conclusion is that clean accuracy does not predict robustness to sensor loss, since the most accurate model degrades the fastest. This is consistent with the aggregation operators, max-aggregation being the most exposed to the zeroed embedding a dropped node propagates, whereas mean and normalized-sum dilute it (Section 5), and it motivates the two mitigations of Sections 3.5 and 3.6.

4.3. Effect of Dropout-Aware Training

Table 6 compares four ways to cope with sensor loss on the identical sweep, all on the RGSAGE-Max backbone: standard training (dropped nodes zeroed), an impute-then-infer baseline, dropout-aware training, and the availability-gated SDR-STGNN; the SDR-STGNN curve is also plotted in Fig. 3.

Imputation baseline: A natural alternative to a training-time fix is to reconstruct dropped sensors before inference. As a lightweight control, the no-training analogue of graph-based imputation [2], we fill each dropped node's three voltage channels with the per-timestep mean of its *available* undirected 1-hop neighbours and present the result to the standard-trained model as observed (mask set to 1, the most favourable setting for imputation); dropped nodes are selected exactly as in the zeroing sweep, so the only change is the fill. Imputation gains little over zeroing (AUDC 0.402 → 0.476; $k = 10$ retention 5.7% → 11.8%) and stays far below dropout-aware training (AUDC 0.913), because the reconstructed values are still fed to a model never trained to distrust them.

Dropout-aware training raises the AUDC of RGSAGE-Max from 0.402 to 0.913 and its $k = 10$ retention from 5.7% to 80.3%: where the standard model collapses to a macro-F1 of 0.048 at 40% sensor loss, the dropout-aware variant still reaches 0.674 ± 0.017, an absolute gain of 0.626 macro-F1 points. Clean

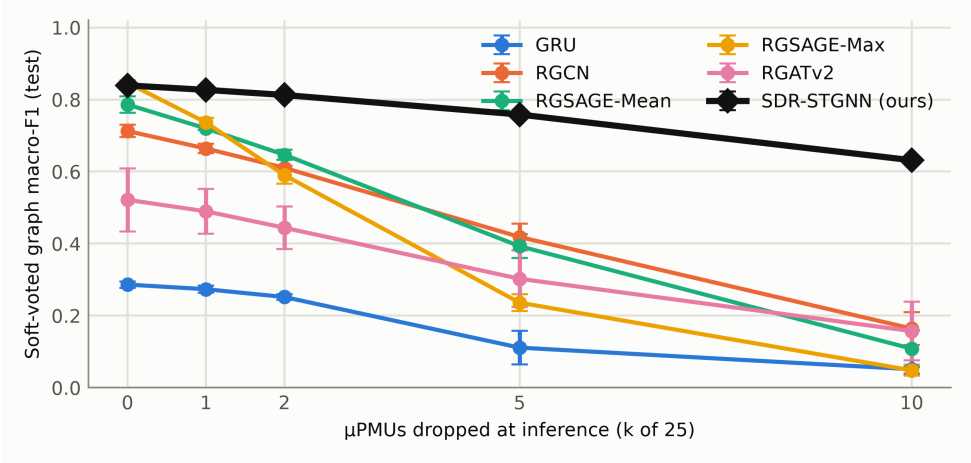


Fig. 3. Soft-voted macro-F1 against the number k of dropped μPMUs for the five baselines and SDR-STGNN (black), with error bars of ±1 std over seeds 42/43/44.

Table 6. Test-time and training-time mitigations on the RGSAGE-Max backbone, as mean ± std over seeds 42/43/44, with the best value per column in bold.

Variant	k=0	k=2	k=5	k=10	AUDC	Ret.@10
Standard training (zeroing)	**0.846±0.007**	0.590±0.024	0.236±0.023	0.048±0.015	0.402	5.7%
Mean-imputation (standard)	0.846±0.006	0.637±0.023	0.336±0.035	0.099±0.027	0.476	11.8%
Dropout-aware training	0.839±0.011	**0.815±0.011**	**0.774±0.013**	**0.674±0.017**	**0.913**	**80.3%**
SDR-STGNN (gated)	0.839±0.002	0.813±0.004	0.758±0.006	0.631±0.010	0.892	75.2%

performance is essentially unchanged (0.839 ± 0.011 against 0.846 ± 0.007 at $k = 0$, within one standard deviation), and because the augmentation alters only the training data it adds no parameters and no inference-time cost (Section 3.5).

The availability-gated SDR-STGNN matches that robustness but does not exceed it: its AUDC is 0.892 and its $k = 10$ retention 75.2%, both slightly below the ungated model, and its curve lies below the dropout-aware curve for all $k \geq 1$. At $k = 0$ the gate of Eq. (10) is inactive, so the architecture reduces exactly to its backbone and the two dropout-aware variants score identically (0.839, within seed noise). The structural gate thus adds nothing beyond the training-time augmentation, leaving dropout-aware training the most effective and most practical mitigation evaluated here.

4.4. Dropout-Aware Training Across Architectures

To test whether the gain is specific to the RGSAGE-Max backbone, we retrain every baseline with the identical augmentation and re-evaluate under the same sweep; the resulting values appear in the "DA" columns of Table 5. Robustness improves for all five architectures. The four spatial GNNs rise from AUDC 0.40–0.62 to 0.91–0.96 and from at most 30% to 80–91% retention at $k = 10$, while clean accuracy changes little: within 0.01 for RGSAGE-Max, and a ~ 0.1 drop for RGCN and RGSAGE-Mean that is the expected cost of training under masking. The GNN-free GRU also improves (AUDC 0.52 $\rightarrow$ 0.78) but its retention stays limited (26%), since without a spatial mechanism it cannot exploit surviving sensors. Robustness recovery is therefore a general property of the training scheme, strongest where the architecture can route information around a failure.

5. Discussion

5.1. Robustness Ranking of the Baseline Architectures

The five baselines expose an inverse relationship between clean accuracy and robustness: the accuracy ranking (RGSAGE-Max > RGSAGE-Mean > RGCN > RGATv2 > GRU) is essentially reversed by the robustness (AUDC) ranking (Tables 4–5). The firm conclusion is the inversion at the extremes: the strongest clean model degrades the fastest, retaining only 5.7% at $k = 10$ against RGCN's 23.1%. The ordering *among* the mid-tier baselines is indicative rather than established, because the spatial depth was selected on RGSAGE-Max and then applied unchanged to every architecture (Section 4.1); a per-architecture depth could plausibly reorder them, and would in particular raise RGATv2, whose high relative retention rests on a clean score depressed by instability at that depth.

This ordering follows from how each operator treats the zeroed embedding a dropped sensor propagates (Section 3.4). The max pooling of Eq. (3) forwards the single largest activation in a neighbourhood, so one corrupted message can dominate a live node's update, making RGSAGE-Max the most fragile despite being the strongest clean model, whereas the normalized sum of Eq. (2) and mean pooling average that message against surviving ones and degrade more gracefully. Attention (Eq. (4)) could in principle down-weight a dropped neighbour, but its coefficients are computed from features and are not availability-aware. GRU performs no spatial aggregation, so it is insensitive to neighbours' dropout but capped by the absence of spatial reasoning.

The aggregation operator is therefore a weak lever for robustness: even the most robust baseline retains under a third of its accuracy at 40% dropout. Deployment-level robustness comes from training instead. Dropout-aware training raises retention from 5.7% to 80.3% on the RGSAGE-Max backbone (Table 6) and, altering only the input masking, applies unchanged to every architecture (Table 5, DA columns). SDR-STGNN confirms this from the opposite side: silencing every dropped node at every hop also removes the multi-hop paths that would otherwise route information around a failure, so the gate adds nothing over the augmentation. Practically, on this feeder an operator could lose roughly 10 of 25 μPMUs to maintenance or communication faults while macro-F1 stays above 0.67, provided the model is trained with the augmentation, which supports resilient monitoring of distribution feeders (relevant to UN SDG 7 and SDG 11) without additional sensing hardware.

5.2. Limitations

Six limitations bound these findings. All experiments use the IEEE 123-bus feeder in its default configuration, so generalization across network sizes and topologies, including the reconfigured ("green") topology of [1], remains open (Section 3.3). Our missingness model drops sensors independently and uniformly at random per window, whereas real μPMU failures are often correlated (co-located units, a shared communication path, or a substation outage fail together) and persistent, so structured failures may degrade, and respond to mitigation, differently. Our 1.65 M-window corpus yields a strongest-baseline macro-F1 of 0.846, not directly comparable to [1], which uses a different corpus (Section 3.3). The four message-passing layers were chosen from a single-seed depth sweep on RGSAGE-Max and then held fixed for all five architectures; combined with RGATv2's seed variance (± 0.087 at $k = 0$), this makes the cross-architecture orderings in Tables 4 and 5 indicative rather than firm (Section 4.2). Our imputation comparison covers only lightweight graph mean-imputation (Section 4.3), not learned reconstruction such as the Dirichlet/contrastive pipeline of [2] or the VGAE of [3], so whether a stronger imputer could close the gap is unresolved. Finally, dropout-aware training samples $k \in \{0, 1, 2, 5\}$, so the $k = 10$ evaluation probes beyond the training distribution; recovery remains strong there, but larger or adaptive dropout budgets were not tuned.

6. Conclusion and Future Work

Building on the STGNN fault-localization methodology of [1], this work added a systematic evaluation of robustness to μPMU sensor loss. Clean accuracy proved an unreliable indicator of robustness: the highest-performing baseline under full observability degraded the fastest as sensors were removed. Our

principal contribution, dropout-aware training, recovered most of the lost performance, raising the retained macro-F1 at 40% sensor dropout from 5.7% to 80.3% without sacrificing clean accuracy or increasing inference cost. The availability-gated SDR-STGNN achieved comparable, but not superior, robustness, indicating that training-time augmentation is a simpler and more effective solution than architectural modification. Future work will investigate correlated and spatially structured sensor failures, larger and more diverse feeder datasets, cross-topology generalization to reconfigured feeders, and adaptive availability-gating mechanisms.

Acknowledgements

The authors thank the Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore, Amrita Vishwa Vidyapeetham, for the institutional support that made this work possible.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) in order to draft and edit manuscript text, correct grammar and improve readability, resolve LaTeX formatting and structural questions, and organise the presentation of the paper. All experimental design, simulation, training, and evaluation were carried out by the authors, and every reported number, table, and figure was produced by the authors' own experimental pipeline rather than generated or altered by the tool. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] Karabulut, Burak, Carlo Manna, and Chris Develder. (2026) "Robustness of spatio-temporal graph neural networks for fault location in partially observable distribution grids." *arXiv preprint* **arXiv:2604.20403**.

[2] Zhang, L., D. Hua, T. Ji, T. Qian, and J. Wang. (2025) "Fault location in power distribution networks with massive missing data: a graph-based imputation and contrastive learning approach." *IEEE Transactions on Automation Science and Engineering* **22**: 16825–16837.

[3] Fan, M., J. Xia, H. Zhang, and X. Zhang. (2024) "Fault location method of distribution network based on VGAE-GraphSAGE." *Processes* **12** (10): 2179.

[4] Krishnathevar, R., and E. E. Ngu. (2012) "Generalized impedance-based fault location for distribution systems." *IEEE Transactions on Power Delivery* **27** (1): 449–451.

[5] Liu, F., L. Xie, K. Yu, Y. Wang, X. Zeng, L. Bi, et al. (2023) "A novel fault location method based on traveling wave for multi-branch distribution network." *Electric Power Systems Research* **224**: 109753.

[6] Livani, H., C. Y. Evrenosoglu, and V. A. Centeno. (2013) "A machine learning-based faulty line identification for smart distribution network." *North American Power Symposium (NAPS)*: 1–6.

[7] Chen, K., J. Hu, Y. Zhang, Z. Yu, and J. He. (2020) "Fault location in power distribution systems via deep graph convolutional networks." *IEEE Journal on Selected Areas in Communications* **38** (1): 119–131.

[8] Nguyen, B. L. H., T. Vu, T.-T. Nguyen, M. Panwar, and R. Hovsapian. (2022) "1-D convolutional graph convolutional networks for fault detection in distributed energy systems." *IEEE 1st Industrial Electronics Society Annual On-Line Conference (ONCON)*: 1–6.

[9] Ngo, Q.-H., B. L. H. Nguyen, J. Zhang, K. Schoder, H. Ginn, and T. Vu. (2025) "Deep graph neural network for fault detection and identification in distribution systems." *Electric Power Systems Research* **247**: 111721.

[10] Karabulut, Burak, Carlo Manna, and Chris Develder. (2025) "Generalization of graph neural network models for distribution grid fault detection." *arXiv preprint* **arXiv:2510.03571**.

[11] Hamilton, Will, Zhitao Ying, and Jure Leskovec. (2017) "Inductive representation learning on large graphs." *Advances in Neural Information Processing Systems* **30**: 1024–1034.

[12] Brody, Shaked, Uri Alon, and Eran Yahav. (2022) "How attentive are graph attention networks?" *International Conference on Learning Representations (ICLR)*.

[13] Rong, Yu, Wenbing Huang, Tingyang Xu, and Junzhou Huang. (2020) "DropEdge: Towards deep graph convolutional networks on node classification." *International Conference on Learning Representations (ICLR)*.